

SALON MANAGEMENT PLATFORM

Technical Implementation Guide

Web Application • React + Spring Boot + PostgreSQL

A build guide written for the intern engineering team

Prepared by	MaverickIgnite Solutions LLP
Document for	Intern Engineering Team
Product	Salon Management Platform (Web)
Stack	React • Spring Boot (Java 17+) • PostgreSQL
Version	1.0
Date	May 2026

Contents

Contents	2
1. How to Read This Document.....	4
2. Product Overview	4
2.1 What we are building (web-only scope)	4
2.2 The core daily flow	4
3. User Roles & Access	5
4. Technology Stack.....	5
5. System Architecture	6
5.1 Backend request layering	6
5.2 Key cross-cutting rules.....	6
6. Functional Modules	8
6.1 Appointment Management.....	8
6.2 Customer (CRM) Management.....	8
6.3 Service Management	9
6.4 Staff Management.....	9
6.5 Billing & POS System.....	9
6.6 Inventory Management	10
6.7 Membership & Loyalty Programs.....	10
6.8 Marketing & Promotions.....	11
6.9 Reports & Analytics	11
6.10 Notifications & Reminders.....	11
7. Database Design (PostgreSQL)	13
7.1 How the tables relate (overview)	13
7.2 Users, Roles & Staff.....	13
7.3 Customers	14
7.4 Service Catalogue	14
7.5 Appointments.....	15
7.6 Billing	16
7.7 Inventory	17
7.8 Membership, Loyalty, Marketing & Notifications	17
8. REST API Design.....	19
8.1 Conventions.....	19
8.2 Standard response envelope	19
8.3 HTTP status codes to use.....	19
8.4 Representative endpoints by module	20
9. Backend Structure (Spring Boot).....	21
9.1 Package layout.....	21

9.2 Entity pattern	21
9.3 Repository pattern	21
9.4 Service pattern (business logic lives here)	21
9.5 Controller pattern (thin — no business logic)	22
9.6 DTOs and validation	22
9.7 One global error handler	22
10. Frontend Structure (React)	23
10.1 Folder layout	23
10.2 One Axios client with the JWT interceptor	23
10.3 Per-feature API module	23
10.4 Fetching data with React Query	23
10.5 Role-based protected routes	23
10.6 Frontend rules	24
11. Authentication & Authorization	25
11.1 Login flow	25
11.2 Security rules	25
12. Local Development Setup	25
12.1 Install once	25
12.2 Backend — first run	25
12.3 Frontend — first run	25
13. Coding Standards & Git Workflow	26
13.1 Naming	26
13.2 General rules	26
13.3 Git workflow	26
13.4 Testing (lightweight for the internship)	26
14. Suggested Build Order	27
15. Definition of Done (per module)	27
16. Glossary	27

1. How to Read This Document

This is a build guide. It tells you what the Salon Management Platform is, what it must do, and exactly how the pieces fit together so you can start coding with confidence. You do not need prior salon-industry knowledge — every term is explained.

The original product was scoped as a Web App + Mobile App. For this internship build we are doing the web application only. Everything the customer or therapist would have done on a phone is now done inside the same web app through different login roles. There is no Flutter / Android / iOS work in this document.

Suggested reading order

1. Read sections 2–5 once to understand the product, the users, and the architecture.
2. Read section 6 (Functional Modules) to learn what each part must do.
3. Keep sections 7 (Database) and 8 (APIs) open while you code — they are your daily reference.
4. Follow section 9 and 10 for the exact backend and frontend folder layout and code patterns.
5. Use section 14 (Build Order) to decide what to build first, and section 15 (Definition of Done) to know when a module is finished.

Rule of thumb: if you are ever unsure how something should behave, re-read the relevant module in section 6. If you are unsure how to structure code, copy the patterns in sections 9 and 10.

2. Product Overview

The Salon Management Platform is a single web application that runs the day-to-day operations of a salon: booking appointments, managing customers, billing them, tracking products, running loyalty programs, and producing reports. One React frontend talks to one Spring Boot backend, which stores everything in one PostgreSQL database.

Because everyone uses the same system at the same time, data stays consistent in real time. When the front desk books an appointment, the assigned stylist sees it. When a stylist marks a service complete, the bill at the front desk updates and the customer's loyalty points are credited — all from the same database.

2.1 What we are building (web-only scope)

Area	What it means for this build
Single web app	One React application. Different menus and pages appear depending on who logs in (role-based access).
One backend	One Spring Boot service exposing REST APIs over HTTPS, secured with JWT tokens.
One database	One PostgreSQL database holding all salon data.
No mobile app	The phone app is out of scope. The therapist's "start / complete service" screens and the customer-facing actions become web pages used by staff.
Customer self-booking	Optional and treated as a later phase. For the core build, the front desk creates and manages bookings. (A public booking page can be added once the core is stable.)

2.2 The core daily flow

Keep this flow in your head — almost every module touches it:

1. Front desk books an appointment for a customer (or registers a walk-in).
2. The appointment is assigned to a stylist/therapist and given a status.
3. On the day, the front desk checks the customer in.
4. The assigned stylist opens their “My Appointments” page, marks the service Started, then Completed, and adds any extra items used during the session.
5. The front desk generates a GST-compliant bill, applies discounts / loyalty, and takes payment.
6. Loyalty points are credited, stylist commission is recorded, inventory is reduced, and a receipt is produced.

3. User Roles & Access

Every staff member logs in with an email and password. Their role decides which menus and APIs they can use. Roles are checked on the backend (never trust the frontend alone).

Role	Who they are	What they can do
OWNER / ADMIN	Salon owner or system administrator	Everything: all reports, P&L, configuration, staff management, and every module.
MANAGER	Branch manager	Day-to-day operations: scheduling, inventory, marketing campaigns, most reports. Cannot change system-wide configuration.
FRONT_DESK	Receptionist / cashier	Bookings, walk-ins, customer check-in, billing and payments, viewing customer profiles.
THERAPIST	Stylist / therapist	Sees only their own assigned appointments; marks services Started / Completed; adds add-ons used; views own performance and commission.

Start simple: store the role as a single column on the user. You do not need a full permissions engine for the internship build — role-based menu visibility plus backend role checks are enough.

4. Technology Stack

Use these exact technologies and versions so everyone's environment matches. Do not introduce other frameworks without asking your mentor.

Layer	Technology	Notes
Frontend	React 18 (with Vite)	Single-page app. Use Vite for fast local dev.
UI styling	Tailwind CSS	Utility-first CSS. No separate CSS framework.
Routing	React Router v6	Role-based protected routes.
Data fetching	Axios + React Query (TanStack Query)	Axios for HTTP; React Query for caching/loading/error state.
Backend	Spring Boot 3.x (Java 17+)	REST APIs. Maven build.
Security	Spring Security + JWT	Stateless auth with JSON Web Tokens.
Data layer	Spring Data JPA (Hibernate)	Entities map to PostgreSQL tables.

Layer	Technology	Notes
Migrations	Flyway	Versioned SQL scripts create/alter tables. Never edit the DB by hand.
Validation	Jakarta Bean Validation	@NotNull, @Size, etc. on request DTOs.
Database	PostgreSQL 15 or 16	Relational, ACID-compliant.
API docs	springdoc-openapi (Swagger UI)	Auto-generated API explorer at /swagger-ui.html.
Build / VCS	Maven, Git (GitHub)	Feature-branch workflow with pull requests.

WhatsApp / SMS / email sending and payment-gateway integrations are kept behind simple interfaces (see section 6.10). For the internship build you can stub them (log to console / mark as SENT) and wire the real providers later.

5. System Architecture

This is a classic three-tier web application. Requests flow in one direction and responses flow back:



5.1 Backend request layering

Every backend feature follows the same three layers. Never skip a layer — controllers must not talk to the database directly.

Layer	Responsibility
Controller	Receives the HTTP request, validates input (DTOs), calls a service, returns a response. No business logic here.
Service	Holds the business rules (e.g. “prevent double-booking”, “calculate GST”, “credit loyalty points”). Coordinates one or more repositories. This is where the real work happens.
Repository	Reads and writes the database via Spring Data JPA. One repository per entity. No business logic.

5.2 Key cross-cutting rules

- All money is stored as NUMERIC(12,2) in the DB and BigDecimal in Java — never use float/double for money.
- All timestamps are stored in UTC (TIMESTAMP TZ). Convert to local time only for display.
- Every table has id (BIGINT, generated), created_at, and updated_at. Soft-delete with an is_active flag instead of hard DELETE wherever history matters.
- Every API returns a consistent JSON envelope (see section 8.2) and uses correct HTTP status codes.

- The frontend never trusts itself for security — the backend re-checks the user's role on every protected endpoint.

6. Functional Modules

These are the ten modules you will build. For each one: read “What it does” to understand the goal, use the capability table as your checklist, and follow the workflow notes for behaviour. Mobile-only features from the original proposal have been folded into web roles.

6.1 Appointment Management

What it does: a single booking calendar that handles pre-booked appointments and walk-ins. The front desk manages everything; stylists see only their own bookings.

Capability	Description
Booking calendar	Day / week / month views with colour-coded statuses.
Create booking	Front desk picks a customer, one or more services, a date, a time slot, and a stylist.
Walk-in registration	One-click instant appointment for someone who just walked in.
Slot management	Slot length is auto-calculated from the total service duration. The system must prevent double-booking the same stylist.
Reschedule / cancel	Move or cancel a booking; status and history are kept.
Status tracking	Booked → Confirmed → Checked-in → In-Service → Completed → Billed.
Stylist view	A therapist logs in and sees only today's appointments assigned to them.

Workflow

1. Front desk creates the appointment and assigns a stylist; status = CONFIRMED.
2. On arrival, front desk marks CHECKED_IN.
3. The stylist opens “My Appointments”, marks the service STARTED (status = IN_SERVICE), then COMPLETED.
4. Front desk bills the appointment (status = BILLED).

Key rule to implement: when assigning a stylist to a slot, reject the save if that stylist already has an overlapping appointment.

6.2 Customer (CRM) Management

What it does: keeps a complete profile of every customer — contact details, full visit history, preferences, and lifetime spend. The profile is the centre of personalization and loyalty.

Data captured	Details
Personal info	Name, mobile (unique), email, gender, date of birth, anniversary.
Visit history	Every appointment: services, stylist, amount paid, date/time. Built automatically from bookings and bills.
Preferences	Preferred services, preferred stylist, allergies / sensitivities, free-text notes.
Loyalty / membership	Current tier, points balance, redemption history.
Lifetime value	Total visits, total spend, average ticket size, last visit date (computed).

Workflow

- A profile is auto-created on first booking or first walk-in (look up by mobile number first to avoid duplicates).

- Visit history, payments, and feedback are linked to the profile automatically — interns do not enter these by hand; they are side effects of other modules.

6.3 Service Management

What it does: the master catalogue of everything the salon sells as a service. It feeds booking (durations), billing (prices, tax) and reports.

Capability	Description
Service menu	Hierarchical: Category → Sub-category → Service (e.g. Hair → Cut → Men's Haircut).
Pricing	Base price, member price, and weekend/festival price configurable per service.
Duration	Standard minutes per service; used to auto-size booking slots.
Combo packages	Bundles of services at a discounted package price (e.g. Bridal Package).
Add-ons	Optional extras a stylist can attach during a session.
Tax	GST % configurable per service or category.
Active / inactive	Disable a service without deleting history (soft flag).

6.4 Staff Management

What it does: manages staff profiles, shifts, attendance, performance and commission. Staff records are the same user accounts used for login.

Module	Details
Staff profiles	Personal details, role, specialization, joining date.
Shift scheduling	Weekly shifts, off-days, leaves.
Attendance	Web-based check-in / check-out; auto working-hours tally.
Performance	Services performed, revenue generated, repeat customers, average rating.
Commission	Configurable rules (see below).
Payroll inputs	Auto-generated commission + attendance reports for payroll.

Commission models to support

- Flat percentage on every service (e.g. 10% of every bill).
- Tiered slabs by monthly revenue (e.g. 8% up to ₹50K, 12% above).
- Target-based bonus (e.g. +₹5,000 if monthly target met).
- Service-specific rate (e.g. 15% on hair colour, 10% on haircut).

Implementation tip: model a commission rule with a type field and a small JSON/columns config. Compute commission at billing time and store the computed amount so reports are fast and stable.

6.5 Billing & POS System

What it does: turns a completed appointment into a single GST-compliant invoice covering services, add-ons and any retail products, then collects payment in one or more modes.

Billing workflow

1. Stylist completes the service and adds any extra items used.
2. System creates a draft bill with all services + add-ons + applicable GST.

3. Front desk applies membership discount, loyalty redemption, or a coupon.
4. Customer pays — Cash, UPI, Card, or split across modes.
5. A GST invoice is generated (printable / PDF) and can be shared on WhatsApp / email.
6. Loyalty points are credited; stylist commission is recorded; stock is reduced.

Capability	Description
Invoice generation	GST-compliant invoice with salon details, GSTIN, HSN/SAC codes, sequential invoice number.
Payment modes	Cash, UPI, Card, Net Banking, Wallet, and split payment across modes.
Discount engine	Coupon codes, membership discounts, manual override (manager-approved).
Hold / park bill	Save a partial bill and resume later.
Refund / void	Refund with audit trail and approval.
Receipt formats	Thermal (58/80mm), A4 PDF, WhatsApp / email delivery.

Money rules: compute line totals as quantity × unit price, apply discounts, then add GST on the discounted amount. Store every line and every tax value — do not recompute totals on the fly in reports.

6.6 Inventory Management

What it does: tracks every product — both retail products sold to customers and consumables used during services — with live stock levels, low-stock alerts and supplier records.

Capability	Description
Product master	SKU, brand, category, purchase price, MRP, sale price, GST rate.
Stock tracking	Live stock updated on every consumption and retail sale (via stock movements).
Usage per service	Map products to services (e.g. a colour service uses 50ml dye); stock auto-reduces at billing.
Low-stock alerts	Configurable reorder level triggers an alert to the manager.
Supplier management	Supplier directory, purchase orders, delivery and payment status.
Stock audit	Periodic physical count with variance report.
Wastage tracking	Log expired/damaged stock with a reason.

Design tip: never edit current stock directly. Insert a stock_movement row (PURCHASE / SALE / CONSUMPTION / WASTAGE / ADJUSTMENT) and let stock be the running total. This gives you a full audit trail for free.

6.7 Membership & Loyalty Programs

What it does: drives repeat visits through membership tiers, points-based rewards and automatic offers.

Program	Details
Membership plans	Tiers (Silver / Gold / Elite) with prepaid wallet, fixed discount, or session packages.
Points & rewards	Earn points per ₹100 spent; redeem against future bills at a configurable rate.
Discounts & offers	Tier-based discount auto-applied at billing.

Program	Details
Validity	Expiry tracking, renewal reminders, grace periods.
Referral rewards	Bonus points / discount when a referred friend visits.
Birthday / anniversary	Auto-generated offer on milestone dates.

6.8 Marketing & Promotions

What it does: run targeted campaigns, segment customers and send personalised offers via SMS, WhatsApp and email. For the internship build, sending can be stubbed behind a provider interface.

Capability	Description
Segmentation	Filter customers by visit frequency, last visit, services availed, spend tier.
WhatsApp / SMS / email	Bulk send using approved templates. Track sent / delivered / read / converted counts.
Coupon codes	Single- or multi-use coupons with validity, value, and usage rules.
Auto greetings	Scheduled birthday / anniversary wishes with optional coupons.
Campaign analytics	Per-campaign delivery and conversion metrics.

Messaging and gateway charges are usage-based and out of scope for billing logic — just record what was sent.

6.9 Reports & Analytics

What it does: operational, financial and performance reports, exportable to Excel / PDF.

Report	What it shows
Daily sales	Total bills, gross revenue, taxes, payment-mode split, discounts given.
Profit & loss	Revenue minus product cost minus salaries/commissions = net profit per period.
Staff performance	Services done, revenue, average rating, commission earned per staff.
Top services / products	Ranking by revenue, units sold, repeat purchases.
Customer retention	New vs returning customers, churn, retention cohorts.
Membership	Active memberships, upcoming expiries, renewal rate, redemption value.
Inventory	Stock movement, low-stock items, fast/slow movers, wastage.
Tax / GST	GST-wise breakdown ready for filing (B2C / B2B summary).

Build reports as read-only SQL queries / aggregate endpoints. Do not change any data in a report endpoint.

6.10 Notifications & Reminders

What it does: an automated, multi-channel engine that keeps customers and staff informed.

Channels: WhatsApp, SMS, email, and in-app alerts.

Notification	When it is sent
Appointment confirmation	Immediately after the front desk confirms a booking.
Appointment reminder	24 hours and 1 hour before the appointment.
Service completion	Bill summary + feedback request after billing.
Payment reminder	For pending dues.

Notification	When it is sent
Membership renewal	Before / on / after expiry.
Birthday / anniversary	Personalised wishes with optional offer.
Staff alerts	Schedule changes, new appointments, low-stock alerts to managers.

Implement a single NotificationService with one method like send(channel, recipient, template, data). Behind it, start with a console/log sender; swap in real WhatsApp/SMS/email senders later without changing callers.

7. Database Design (PostgreSQL)

This is your schema reference. Use snake_case for table and column names. Every table has id BIGINT GENERATED ALWAYS AS IDENTITY plus created_at and updated_at TIMESTAMPTZ (omitted in the lists below to save space — add them everywhere). Create all tables through Flyway migration scripts, never by hand.

7.1 How the tables relate (overview)

```

users (staff) ---< staff_shifts, attendance, commission_rules

customers ---< appointments ---< appointment_services >--- services
|                                     |
|                                     v                                     >--- users (assigned staff)
|                                     |
|                                     invoices ---< invoice_items
|                                     |
|                                     >--- payments
|
|--- customer_memberships >--- membership_plans
|--- loyalty_transactions

products ---< stock_movements          service_consumption (service<->product)
products >--- suppliers (via purchase_orders ---< purchase_order_items)

campaigns, coupons ---< coupon_usages, feedback, notifications

```

Read “---<” as “has many”. Example: one customer has many appointments; one appointment has many appointment_services.

7.2 Users, Roles & Staff

users

Column	Type / Note
full_name	VARCHAR(150) NOT NULL
email	VARCHAR(150) UNIQUE NOT NULL — login id
phone	VARCHAR(20)
password_hash	VARCHAR(255) NOT NULL — BCrypt hash, never plain text
role	VARCHAR(20) NOT NULL — OWNER / MANAGER / FRONT_DESK / THERAPIST
specialization	VARCHAR(120) — e.g. “Hair Colour” (for stylists)
joining_date	DATE
is_active	BOOLEAN NOT NULL DEFAULT true

staff_shifts

Column	Type / Note
user_id	BIGINT FK → users(id)
day_of_week	SMALLINT — 1 (Mon) .. 7 (Sun)
start_time	TIME
end_time	TIME

attendance

Column	Type / Note
user_id	BIGINT FK → users(id)
work_date	DATE NOT NULL
check_in	TIMESTAMPTZ
check_out	TIMESTAMPTZ

commission_rules

Column	Type / Note
user_id	BIGINT FK → users(id) — nullable if rule is global
service_id	BIGINT FK → services(id) — nullable; for service-specific rates
rule_type	VARCHAR(20) — FLAT_PERCENT / TIERED / TARGET / SERVICE_SPECIFIC
config	JSONB — slabs, target amount, bonus, etc.
is_active	BOOLEAN DEFAULT true

7.3 Customers**customers**

Column	Type / Note
full_name	VARCHAR(150) NOT NULL
mobile	VARCHAR(20) UNIQUE NOT NULL — used to find existing customers
email	VARCHAR(150)
gender	VARCHAR(10)
date_of_birth	DATE
anniversary	DATE
preferred_stylist_id	BIGINT FK → users(id), nullable
allergies	TEXT
notes	TEXT
loyalty_points	INTEGER NOT NULL DEFAULT 0
is_active	BOOLEAN DEFAULT true

feedback

Column	Type / Note
appointment_id	BIGINT FK → appointments(id)
customer_id	BIGINT FK → customers(id)
rating	SMALLINT — 1..5
comments	TEXT

7.4 Service Catalogue**service_categories**

Column	Type / Note
name	VARCHAR(120) NOT NULL
parent_id	BIGINT FK → service_categories(id), nullable — self-reference for sub-categories

services

Column	Type / Note
category_id	BIGINT FK → service_categories(id)
name	VARCHAR(150) NOT NULL
description	TEXT
base_price	NUMERIC(12,2) NOT NULL
member_price	NUMERIC(12,2)
weekend_price	NUMERIC(12,2)
duration_minutes	INTEGER NOT NULL — used for slot length
gst_percent	NUMERIC(5,2) NOT NULL DEFAULT 18
is_active	BOOLEAN DEFAULT true

service_packages & package_items

Column	Type / Note
service_packages.name	VARCHAR(150) NOT NULL
service_packages.package_price	NUMERIC(12,2) NOT NULL
service_packages.gst_percent	NUMERIC(5,2) NOT NULL
package_items.package_id	BIGINT FK → service_packages(id)
package_items.service_id	BIGINT FK → services(id)

7.5 Appointments

appointments

Column	Type / Note
customer_id	BIGINT FK → customers(id)
appointment_date	DATE NOT NULL
start_time	TIME NOT NULL
end_time	TIME NOT NULL — start + total service duration
status	VARCHAR(20) — BOOKED / CONFIRMED / CHECKED_IN / IN_SERVICE / COMPLETED / BILLED / CANCELLED
source	VARCHAR(15) — WALK_IN / ONLINE / FRONT_DESK
notes	TEXT
created_by	BIGINT FK → users(id)

appointment_services

One row per service (or add-on) within an appointment. This is what the stylist starts and completes.

Column	Type / Note
appointment_id	BIGINT FK → appointments(id)
service_id	BIGINT FK → services(id)
assigned_user_id	BIGINT FK → users(id) — the stylist
price	NUMERIC(12,2) — price captured at booking time
status	VARCHAR(15) — PENDING / STARTED / COMPLETED
is_addon	BOOLEAN DEFAULT false — true if added during the session
started_at / completed_at	TIMESTAMPTZ, nullable

7.6 Billing

invoices

Column	Type / Note
invoice_number	VARCHAR(30) UNIQUE NOT NULL — sequential, e.g. INV-2026-000123
appointment_id	BIGINT FK → appointments(id), nullable (retail-only sale)
customer_id	BIGINT FK → customers(id)
subtotal	NUMERIC(12,2) NOT NULL
discount_amount	NUMERIC(12,2) NOT NULL DEFAULT 0
tax_amount	NUMERIC(12,2) NOT NULL DEFAULT 0
total_amount	NUMERIC(12,2) NOT NULL
payment_status	VARCHAR(15) — DRAFT / PARKED / PAID / PARTIALLY_PAID / REFUNDED / VOID
created_by	BIGINT FK → users(id)

invoice_items

Column	Type / Note
invoice_id	BIGINT FK → invoices(id)
item_type	VARCHAR(15) — SERVICE / PRODUCT / PACKAGE / ADDON
reference_id	BIGINT — id of the service/product/package
name	VARCHAR(150) — captured at sale time
quantity	NUMERIC(10,2) NOT NULL DEFAULT 1
unit_price	NUMERIC(12,2) NOT NULL
tax_percent	NUMERIC(5,2) NOT NULL
line_total	NUMERIC(12,2) NOT NULL

payments

Column	Type / Note
invoice_id	BIGINT FK → invoices(id)
payment_mode	VARCHAR(15) — CASH / UPI / CARD / NET_BANKING / WALLET
amount	NUMERIC(12,2) NOT NULL

Column	Type / Note
reference_no	VARCHAR(100) — txn id, nullable
paid_at	TIMESTAMPTZ NOT NULL

Split payment = multiple payments rows for one invoice. The invoice is PAID when the sum of payments equals total_amount.

7.7 Inventory

products

Column	Type / Note
sku	VARCHAR(50) UNIQUE NOT NULL
name / brand / category	VARCHAR
purchase_price / mrp / sale_price	NUMERIC(12,2)
gst_percent	NUMERIC(5,2)
current_stock	NUMERIC(12,2) NOT NULL DEFAULT 0 — running total of movements
reorder_level	NUMERIC(12,2) — low-stock threshold
is_active	BOOLEAN DEFAULT true

stock_movements

Column	Type / Note
product_id	BIGINT FK → products(id)
movement_type	VARCHAR(15) — PURCHASE / SALE / CONSUMPTION / WASTAGE / ADJUSTMENT
quantity	NUMERIC(12,2) — positive in, negative out
reference	VARCHAR(100) — e.g. invoice number or PO number
reason	VARCHAR(150) — required for WASTAGE / ADJUSTMENT

service_consumption

Column	Type / Note
service_id	BIGINT FK → services(id)
product_id	BIGINT FK → products(id)
quantity_used	NUMERIC(12,2) — e.g. 50 (ml) per service

suppliers / purchase_orders / purchase_order_items

Column	Type / Note
suppliers	name, contact_person, phone, email, address
purchase_orders	supplier_id FK, order_date, status (DRAFT/ORDERED/RECEIVED), total_amount, payment_status
purchase_order_items	po_id FK, product_id FK, quantity, unit_price

7.8 Membership, Loyalty, Marketing & Notifications

membership_plans / customer_memberships

Column	Type / Note
membership_plans	name, tier, price, discount_percent, wallet_value, validity_days, is_active
customer_memberships	customer_id FK, plan_id FK, start_date, expiry_date, wallet_balance, status (ACTIVE/EXPIRED)

loyalty_transactions

Column	Type / Note
customer_id	BIGINT FK → customers(id)
txn_type	VARCHAR(10) — EARN / REDEEM
points	INTEGER — positive for earn, negative for redeem
invoice_id	BIGINT FK → invoices(id), nullable

coupons / coupon_usages

Column	Type / Note
coupons	code (UNIQUE), discount_type (PERCENT/FLAT), value, max_uses, used_count, valid_from, valid_to, is_active
coupon_usages	coupon_id FK, customer_id FK, invoice_id FK, used_at

campaigns / notifications

Column	Type / Note
campaigns	name, channel (WHATSAPP/SMS/EMAIL), segment_filter JSONB, template, status, scheduled_at, sent_count, delivered_count
notifications	recipient_type (CUSTOMER/STAFF), recipient_id, channel, notif_type, message, status (PENDING/SENT/FAILED), sent_at

8. REST API Design

All APIs live under the /api base path and exchange JSON. They are stateless — the client sends a JWT in the Authorization header on every protected request.

8.1 Conventions

Action	HTTP method + path	Returns
List (with paging/filter)	GET /api/customers?page=0&size=20&q=...	Page of items
Get one	GET /api/customers/{id}	Single item
Create	POST /api/customers	Created item, 201
Update	PUT /api/customers/{id}	Updated item, 200
Partial update	PATCH /api/customers/{id}	Updated item, 200
Delete / deactivate	DELETE /api/customers/{id}	204 (prefer soft-delete)

- Use plural nouns for collections (/appointments, /invoices). No verbs in paths.
- Use sub-resources for things that belong to a parent (e.g. POST /api/appointments/{id}/services).
- Use query params for filtering, sorting and paging — never put filters in the path.

8.2 Standard response envelope

Every response uses the same shape so the frontend can handle them uniformly:

```
// Success
{
  "success": true,
  "data": { /* object or array */ },
  "message": null
}

// Error
{
  "success": false,
  "data": null,
  "message": "Customer mobile already exists",
  "errors": { "mobile": "must be unique" }
}
```

8.3 HTTP status codes to use

Code	When
200 OK	Successful GET / PUT / PATCH.
201 Created	Successful POST that creates a resource.
204 No Content	Successful DELETE.
400 Bad Request	Validation failed (bad input).
401 Unauthorized	Missing or invalid JWT.
403 Forbidden	Valid user, but role not allowed.
404 Not Found	Resource id does not exist.
409 Conflict	Business conflict (e.g. double-booking, duplicate mobile).

Code	When
500 Server Error	Unexpected — log it; never leak stack traces to the client.

8.4 Representative endpoints by module

This is not exhaustive — it shows the shape. Follow the same pattern for every module.

Module	Endpoint	Purpose
Auth	POST /api/auth/login	Email + password → JWT.
Auth	GET /api/auth/me	Current user's profile + role.
Customers	GET/POST /api/customers	List / create customers.
Customers	GET /api/customers/{id}/history	Visit history + lifetime value.
Services	GET/POST /api/services	Catalogue management.
Services	GET /api/service-categories	Category tree.
Appointments	GET /api/appointments?date=&staffId=	Calendar / my-appointments view.
Appointments	POST /api/appointments	Create booking (checks double-booking).
Appointments	PATCH /api/appointments/{id}/status	Move through the status flow.
Appointments	PATCH /api/appointment-services/{id}/start complete	Stylist starts/completes a service.
Billing	POST /api/invoices/from-appointment/{id}	Build a draft bill from an appointment.
Billing	POST /api/invoices/{id}/payments	Record a payment (supports split).
Inventory	GET /api/products?lowStock=true	Products / low-stock list.
Inventory	POST /api/products/{id}/movements	Record a stock movement.
Loyalty	POST /api/customers/{id}/loyalty/redeem	Redeem points on a bill.
Marketing	POST /api/campaigns / GET /api/coupons	Campaigns and coupons.
Reports	GET /api/reports/daily-sales?date=	Read-only aggregates.

springdoc-openapi auto-publishes a live, clickable list of all your endpoints at /swagger-ui.html. Use it to test as you build.

9. Backend Structure (Spring Boot)

Organize code by feature (package-by-feature), not by layer. Each feature folder contains its own controller, service, repository, entity and DTOs. This keeps related code together.

9.1 Package layout

```
com.maverickignite.salon
|- SalonApplication.java      (main class)
|- config/                   (CORS, OpenAPI, beans)
|- security/                 (JWT filter, SecurityConfig, UserDetails)
|- common/
|   |- ApiResponse.java      (the success/error envelope)
|   |- GlobalExceptionHandler.java
|   |- exception/ (NotFoundException, ConflictException, ...)
|   |- BaseEntity.java       (id, createdAt, updatedAt)
|- auth/                     (login, current user)
|- user/                     (staff)
|- customer/
|- service_catalog/
|- appointment/
|- billing/
|- inventory/
|- membership/
|- marketing/
|- report/
|- notification/
```

Each feature folder, e.g. customer/:

```
CustomerController.java  CustomerService.java
CustomerRepository.java  Customer.java (entity)
dto/ CustomerRequest.java CustomerResponse.java
```

9.2 Entity pattern

```
@Entity @Table(name = "customers")
@Getter @Setter
public class Customer extends BaseEntity {
    @Column(nullable = false) private String fullName;
    @Column(unique = true, nullable = false) private String mobile;
    private String email;
    private LocalDate dateOfBirth;
    @Column(nullable = false) private Integer loyaltyPoints = 0;
    @Column(nullable = false) private boolean active = true;
}
```

9.3 Repository pattern

```
public interface CustomerRepository
    extends JpaRepository<Customer, Long> {
    Optional<Customer> findByMobile(String mobile);
    Page<Customer> findByFullNameContainingIgnoreCase(
        String q, Pageable pageable);
}
```

9.4 Service pattern (business logic lives here)

```
@Service @RequiredArgsConstructor
public class CustomerService {
    private final CustomerRepository repo;
```

```

@Transactional
public CustomerResponse create(CustomerRequest req) {
    repo.findByMobile(req.mobile()).ifPresent(c -> {
        throw new ConflictException("Mobile already exists");
    });
    Customer c = new Customer();
    c.setFullName(req.fullName());
    c.setMobile(req.mobile());
    return CustomerResponse.from(repo.save(c));
}
}

```

9.5 Controller pattern (thin — no business logic)

```

@RestController
@RequestMapping("/api/customers")
@RequiredArgsConstructor
public class CustomerController {
    private final CustomerService service;

    @PostMapping
    @PreAuthorize("hasAnyRole('FRONT_DESK', 'MANAGER', 'OWNER')")
    public ApiResponse<CustomerResponse> create(
        @Valid @RequestBody CustomerRequest req) {
        return ApiResponse.ok(service.create(req));
    }
}

```

9.6 DTOs and validation

Never expose entities directly over the API. Accept a Request DTO and return a Response DTO. Validate the request DTO with annotations.

```

public record CustomerRequest(
    @NotBlank @Size(max = 150) String fullName,
    @NotBlank @Pattern(regexp = "\\d{10}") String mobile,
    @Email String email) {}

```

9.7 One global error handler

Catch exceptions in one place and turn them into the error envelope with the right status code.

```

@RestControllerAdvice
public class GlobalExceptionHandler {
    @ExceptionHandler(NotFoundException.class)
    @ResponseStatus(HttpStatus.NOT_FOUND)
    public ApiResponse<?> notFound(NotFoundException e) {
        return ApiResponse.error(e.getMessage());
    }
    @ExceptionHandler(ConflictException.class)
    @ResponseStatus(HttpStatus.CONFLICT)
    public ApiResponse<?> conflict(ConflictException e) {
        return ApiResponse.error(e.getMessage());
    }
}

```

Golden rule: Controllers are thin, Services are smart, Repositories are dumb. If you are writing an if-statement in a controller, it probably belongs in the service.

10. Frontend Structure (React)

Organize the React app by feature too, mirroring the backend. Shared building blocks (buttons, tables, inputs, layout) live in components/. Each feature has its own folder with its pages, components, and API calls.

10.1 Folder layout

```
src/
|- main.jsx, App.jsx
|- api/
|   |- axiosClient.js    (base URL + JWT interceptor)
|   |- customers.js      (customer API calls)
|- components/           (Button, Table, Modal, Input, Layout, Sidebar)
|- context/ AuthContext.jsx
|- hooks/   useAuth.js, useDebounce.js
|- routes/ AppRoutes.jsx, ProtectedRoute.jsx
|- features/
|   |- auth/             LoginPage.jsx
|   |- dashboard/
|   |   |- appointments/ CalendarPage.jsx, BookingForm.jsx,
|   |   |                       MyAppointmentsPage.jsx
|   |   |- customers/   CustomerListPage.jsx, CustomerForm.jsx
|   |   |- services/    staff/ billing/ inventory/
|   |   |- membership/ marketing/ reports/
|- utils/   formatMoney.js, formatDate.js
```

10.2 One Axios client with the JWT interceptor

```
import axios from 'axios';
const api = axios.create({ baseURL: '/api' });
api.interceptors.request.use((config) => {
  const token = localStorage.getItem('token');
  if (token) config.headers.Authorization = `Bearer ${token}`;
  return config;
});
export default api;
```

10.3 Per-feature API module

```
import api from './axiosClient';
export const getCustomers = (params) =>
  api.get('/customers', { params }).then(r => r.data.data);
export const createCustomer = (body) =>
  api.post('/customers', body).then(r => r.data.data);
```

10.4 Fetching data with React Query

```
function CustomerListPage() {
  const [q, setQ] = useState('');
  const { data, isLoading, isError } = useQuery({
    queryKey: ['customers', q],
    queryFn: () => getCustomers({ q }),
  });
  if (isLoading) return <Spinner />;
  if (isError) return <ErrorBox />;
  return <CustomerTable rows={data} onSearch={setQ} />;
}
```

10.5 Role-based protected routes

```
function ProtectedRoute({ allow, children }) {  
  const { user } = useAuth();  
  if (!user) return <Navigate to="/login" />;  
  if (allow && !allow.includes(user.role))  
    return <Navigate to="/forbidden" />;  
  return children;  
}  
  
// usage  
<Route path="/reports" element={  
  <ProtectedRoute allow={['OWNER', 'MANAGER']}>  
    <ReportsPage />  
  </ProtectedRoute>  
} />
```

10.6 Frontend rules

- Show clear loading, empty, and error states for every screen that fetches data.
- Keep components small; if a file is over ~200 lines, split it.
- All money and dates go through `utils/formatMoney` and `formatDate` — never format inline.
- The sidebar menu only shows items the logged-in role is allowed to see.
- Do NOT store secrets in the frontend. The JWT in `localStorage` is the only credential the browser holds.

11. Authentication & Authorization

Authentication = “who are you” (login). Authorization = “what are you allowed to do” (role checks). We use stateless JWT.

11.1 Login flow

1. User submits email + password to POST /api/auth/login.
2. Backend looks up the user, checks the password against the BCrypt hash.
3. On success, backend signs a JWT containing the user id and role, with an expiry (e.g. 8 hours).
4. Frontend stores the token and sends it as Authorization: Bearer <token> on every request.
5. A Spring Security filter validates the token on each request and loads the user's role for @PreAuthorize checks.

11.2 Security rules

- Passwords are stored only as BCrypt hashes — never plain text, never reversible encryption.
- Every protected endpoint declares the roles allowed (@PreAuthorize). The default is “deny”.
- A therapist's “my appointments” query must filter by their own user id on the backend — do not rely on the frontend to send only their data.
- Tokens expire. The frontend should redirect to login on a 401 response.

12. Local Development Setup

12.1 Install once

Tool	Version / note
JDK	Java 17 or newer
Maven	3.9+ (or use the bundled mvnw wrapper)
Node.js	18 LTS or newer (includes npm)
PostgreSQL	15 or 16, running locally
Git	Latest
IDE	IntelliJ IDEA (backend), VS Code (frontend)

12.2 Backend — first run

```
# 1. create the database
createdb salon_db

# 2. set DB connection in application.yml
#   spring.datasource.url: jdbc:postgresql://localhost:5432/salon_db
#   username / password as per your local Postgres

# 3. run (Flyway auto-creates tables on startup)
./mvnw spring-boot:run

# API docs: http://localhost:8080/swagger-ui.html
```

12.3 Frontend — first run

```
npm install
# set the API proxy / base URL to http://localhost:8080
npm run dev
# app: http://localhost:5173
```

Tip: keep a seed Flyway migration that inserts one OWNER user and a few sample services/customers so everyone can log in and test immediately.

13. Coding Standards & Git Workflow

13.1 Naming

Where	Convention
Java classes	PascalCase (CustomerService)
Java methods / fields	camelCase (createCustomer)
DB tables / columns	snake_case (customer_memberships)
REST paths	lowercase plural nouns (/api/customers)
React components	PascalCase files (CustomerForm.jsx)
React hooks	camelCase starting with use (useAuth)

13.2 General rules

- One feature = one branch = one pull request. Keep PRs small and focused.
- No commented-out code or console.logs left in a PR.
- Money is BigDecimal / NUMERIC; never float. Dates are stored in UTC.
- Write a clear commit message: “feat(billing): add split payment support”.
- If you are stuck for more than ~30 minutes, ask your mentor — do not silently struggle.

13.3 Git workflow

1. git checkout -b feature/customer-crud from the main branch.
2. Commit small, logical changes as you go.
3. Push and open a pull request; request a review.
4. Address review comments, get one approval, then merge.
5. Never push directly to main.

13.4 Testing (lightweight for the internship)

- Backend: write at least a few JUnit tests for service-layer business rules (e.g. double-booking is rejected, GST is calculated correctly).
- Frontend: manually test each screen's loading/empty/error states; add component tests where time allows.
- Use Swagger UI to test every endpoint before wiring it to the frontend.

14. Suggested Build Order

Build in this order. Each phase depends on the ones before it, so resist jumping ahead.

Phase	Module	Why this order
1	Project skeleton + Auth + Users	Nothing works without login, roles, and the base project (envelope, error handler, security).
2	Service catalogue	Bookings and bills need services, prices and durations to exist first.
3	Customers (CRM)	Bookings and bills attach to a customer.
4	Appointments	The core flow: booking, assignment, status, the stylist's "my appointments" view.
5	Billing & POS	Turns completed appointments into invoices and payments.
6	Inventory	Stock movements; auto-reduce stock at billing.
7	Membership & Loyalty	Plug points/discounts into the billing flow.
8	Reports & Analytics	Read-only aggregates over everything built so far.
9	Marketing & Notifications	Campaigns, coupons, and the notification engine (stubbed sender).
10	Polish + (optional) public booking page	Hardening, edge cases, and any stretch goals.

Within each phase, build backend first (entity → repository → service → controller, test in Swagger), then the React screens.

15. Definition of Done (per module)

A module is only "done" when ALL of these are true. Use this as your checklist before opening a pull request.

- Flyway migration(s) created the tables; the app starts cleanly.
- Entity, repository, service and controller exist and follow the patterns in section 9.
- Request DTOs are validated; invalid input returns 400 with field errors.
- Business rules from section 6 are enforced in the service layer (e.g. no double-booking, GST correct).
- Endpoints use the standard envelope and correct status codes, and are protected by the right roles.
- Every endpoint is verified in Swagger UI.
- The React screen(s) handle loading, empty, error and success states.
- Only the allowed roles can see the menu item and reach the screen.
- Code is on a feature branch, reviewed, and approved.

16. Glossary

Term	Meaning
Walk-in	A customer who arrives without a prior booking; the front desk creates an instant appointment.

Term	Meaning
Therapist / Stylist	The staff member who performs the service. Logs in to see and update their own appointments.
Add-on	An extra service or upgrade added during a session (e.g. a hair treatment added to a haircut).
Slot	A reserved block of time on a stylist's calendar, sized by the service duration.
GST	Goods & Services Tax (India). A percentage added to the bill; configurable per service.
HSN / SAC	Tax classification codes printed on a GST invoice.
Loyalty points	Points a customer earns by spending, redeemable against future bills.
Membership tier	A paid plan (e.g. Gold) giving discounts, a prepaid wallet, or session packages.
Stock movement	A single recorded change to product stock (purchase, sale, consumption, wastage, adjustment).
JWT	JSON Web Token — a signed token proving who the logged-in user is and their role.
DTO	Data Transfer Object — the request/response shape exposed by the API (separate from the DB entity).
Flyway migration	A versioned SQL script that creates or changes database tables in a controlled way.
Soft-delete	Marking a row inactive (is_active = false) instead of physically deleting it, to keep history.

— End of Implementation Guide —

MaverickIgnite Solutions LLP • Questions? Ask your mentor.